



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS
BACHARELADO EM TECNOLOGIA DA INFORMAÇÃO
PROGRAMAÇÃO ORIENTADA A OBJETOS
DOCENTE: BRUNO BORGES DA SILVA

ALEX BRUNO DUARTE - 2025022557
HARLEY LUCAS DE SOUZA BATISTA - 2025022526
IZADORA LOUYZA SILVA FIGUEIREDO - 2024010176
LEONARDO AUGUSTO SILVA DE SOUZA - 2023011257
LÍVIAN MARIA LUCENA GOMES PINHEIRO - 2024010084
MARCELO CAAT AMARAL DO NASCIMENTO - 2025011536
MARIA VITÓRIA FERNANDES ROCHA - 2024010257

DOCUMENTAÇÃO DO PROJETO FINAL: GERENCIAMENTO HOSPITALAR

SUMÁRIO

1. DESCRIÇÃO GERAL E FUNCIONALIDADES.....	3
1.1 FUNCIONALIDADES PRINCIPAIS.....	3
2. TABELA DE ENTIDADES E ATRIBUTOS.....	3
3. DESCRIÇÃO DA ARQUITETURA E DECISÕES DE PROJETO.....	5
3.1 ORGANIZAÇÃO DE PACOTES.....	6
3.2 FLUXO GERAL DE OPERAÇÃO.....	6
3.3 REGRAS DE NEGÓCIO.....	8

GERENCIAMENTO HOSPITALAR

1. DESCRIÇÃO GERAL E FUNCIONALIDADES

O projeto consiste em um sistema de gerenciamento hospitalar em Java, executado integralmente via terminal. A aplicação tem como objetivo informatizar e organizar o fluxo de atendimento hospitalar, desde o cadastro de pessoas até o agendamento de consultas médicas. O sistema implementa mecanismos de controle de acesso por meio de autenticação via *login*, garantindo maior segurança, rastreabilidade das ações e integridade das informações manipuladas.

1.1 FUNCIONALIDADES PRINCIPAIS

O sistema foi estruturado com base em sete funcionalidades principais, que compõem o núcleo operacional do projeto:

- Autenticação por *login*: cada usuário possui credenciais individuais, associadas a um nível de acesso específico, o que determina as operações que podem ser executadas no sistema.
- Cadastro de pessoas: permite o cadastramento completo de médicos, funcionários e pacientes, assegurando a padronização e integridade dos dados inseridos no sistema;
- Gerenciamento de consultas: disponibiliza o agendamento e verificação das consultas médicas registradas.
- Listagens gerais: possibilita a listagem de médicos, funcionários, pacientes e consultas, facilitando a administração e o controle das informações.
- Busca específica: realiza busca de médicos, funcionários ou pacientes por ID, além de permitir a busca de consultas filtradas por médico, paciente ou data.
- Remoção e alteração: permite remover qualquer entidade cadastrada ou modificar seus dados por meio dos gerenciadores específicos.
- Povoamento automático: executa a inserção dos dados iniciais para fins de teste, acelerando o processo de validação e uso do sistema em cenários simulados.

2. TABELA DE ENTIDADES E ATRIBUTOS

Tabela 1 – Lista de Entidades e Atributos do Sistema

Classe (Entidade)	Atributos Principais	Descrição
-------------------	----------------------	-----------

Pessoa (Abstrata)	id, nome, cpf, telefone, email, endereco	Superclasse que reúne os atributos comuns às demais entidades e define o contrato de exibição que deve ser implementado pelas classes filhas.
Paciente	idade, principalQueixa	Especializa a classe Pessoa, representando o indivíduo que recebe atendimento médico e possui informações clínicas iniciais relevantes.
Medico	crm, especialidade, credenciais	Herda de Pessoa e inclui dados profissionais específicos, como CRM (com validação de UF) e credenciais de acesso ao sistema.
Funcionario	cargo, setor, credenciais	Também herda de Pessoa e modela os colaboradores administrativos do hospital, contendo dados funcionais e credenciais para autenticação.
Consulta	id, paciente, medico, dataHora	Classe associativa responsável por vincular um médico e um paciente em um horário determinado, compondo o fluxo de atendimentos.
UsuarioSistema	usuario, senha, nivelAcesso	Entidade dedicada exclusivamente à autenticação e autorização, estabelecendo níveis de permissão e mecanismos de <i>login</i> .
Hospital	pessoas, consultas	Estrutura centralizadora que armazena as coleções principais do sistema, facilitando o acesso global pelas camadas de gerenciamento.
Login	usuarios, usuarioLogado	Responsável pelos processos de autenticação, gerenciamento de sessão e verificação de permissões.
Gerenciadores (medicos, pacientes, consultas)	métodos de adicionar, listar, buscar, editar, remover	Conjuntos de classes que encapsulam as operações CRUD das entidades, assegurando organização e modularidade no controle dos registros.
Utilitarios	validações, geração de ID, impressão, menus	Classe de suporte que concentra rotinas auxiliares, como validação de dados, criação de IDs, formatação de saídas e exibição de menus.

3. DESCRIÇÃO DA ARQUITETURA E DECISÕES DE PROJETO

O projeto foi construído sobre os pilares da Orientação a Objetos:

a. Abstração e Herança

A classe abstrata *Pessoa* funciona como a superestrutura conceitual do modelo, impedindo a instanciação direta de um indivíduo genérico. Essa abordagem garante que o sistema opere exclusivamente com entidades especializadas, como *Medico* e *Paciente*, preservando a coerência do domínio. Apesar disso, a hierarquia permite que essas classes sejam tratadas uniformemente como *Pessoa* em operações genéricas, como listagens e buscas globais, explorando o compartilhamento de comportamentos e atributos comuns.

b. Interfaces

- *Gerenciavel<T>*: define um contrato padronizado para todas as classes responsáveis pela gestão de entidades (como *GerenciadorMedico* e *GerenciadorPaciente*). Essa interface assegura que funcionalidades essenciais – adicionar, listar, buscar e remover – sejam implementadas de maneira consistente, favorecendo a padronização e a intercambiabilidade entre gerenciadores.
- *Validavel*: estabelece um conjunto mínimo de regras de validação que toda entidade deve cumprir antes de ser persistida no sistema. Entre as verificações mais relevantes estão a validade do CPF, a consistência da idade e demais restrições de negócio necessárias à integridade dos dados.

c. Polimorfismo

O sistema utiliza polimorfismo de forma explícita na classe *Main*, especialmente no método *buscaGeral*, que percorre uma lista contendo objetos do tipo *Pessoa*. Ao invocar *exibirInformacoes()*, o Java determina em tempo de execução qual implementação deve ser acionada – a de *Médico* ou a de *Paciente* – eliminando a necessidade de estruturas condicionais redundantes.

d. Tratamento de Exceções

Foi criada a exceção personalizada *LoginException*, destinada a lidar exclusivamente com falhas relacionadas à autenticação. Isso permite separar a lógica de detecção e tratamento

de erros da lógica de negócio principal, promovendo clareza, rastreabilidade e robustez no controle de acesso ao sistema.

3.1 ORGANIZAÇÃO DE PACOTES

O sistema foi dividido em pacotes para facilitar manutenção e modularidade:

- `br.com.hospital` – contém a classe *Main*, responsável por inicializar o fluxo principal do sistema, acionar os gerenciadores e coordenar a interação com o usuário via terminal.
- `br.com.hospital.sistema` – contém *Hospital*, *Login* e *UsuarioSistema*, classes essenciais ao funcionamento estrutural da aplicação;
- `br.com.hospital.entidades` – contém *Pessoa*, *Medico*, *Paciente*, *Funcionario* e *Consulta*, classes que modelam as entidades fundamentais;
- `br.com.hospital.gerenciadores` – contém *GerenciadorMedico*, *GerenciadorPaciente* e *GerenciadorConsulta*, gerenciadores responsáveis pelas operações CRUD das entidades principais;
- `br.com.hospital.interfaces` – contém *Gerenciavel* e *Validavel*, interfaces utilizadas como contratos formais;
- `br.com.hospital.utilitarios` – contém *Utilitarios*, responsável por armazenar funções auxiliares como validações, geração de IDs, formatação de saídas e exibição de menus;
- `br.com.hospital.enums` – contém *NivelAcesso*, enum utilizado para padronizar e tipar corretamente os diferentes níveis de permissão dos usuários do sistema;
- `br.com.hospital.exceptions` – contém *LoginException*, criada para tratar erros específicos de autenticação, mantendo a separação entre lógica de negócio e tratamento de falhas;

3.2 FLUXO GERAL DE OPERAÇÃO

Esta seção descreve os principais fluxos operacionais do sistema de gerenciamento hospitalar, detalhando as interações do usuário com as funcionalidades essenciais e os mecanismos internos que asseguram a integridade dos dados e o correto funcionamento da aplicação.

a. Fluxo de Login

Ao iniciar o sistema, o usuário é solicitado a informar seu *login* e senha. As credenciais são verificadas pelo módulo de autenticação, que valida usuário, senha e nível de acesso. Caso os dados estejam corretos, o usuário é direcionado ao menu correspondente ao seu perfil: Administrador, Secretaria ou Médico. Se houver divergência nos dados de autenticação, é lançada uma *LoginException*, isolando a falha do fluxo principal e informando o erro de forma clara. Em seguida, o sistema retorna à etapa de solicitação de credenciais, permitindo uma nova tentativa.

b. Fluxo de Cadastro

Ao selecionar a opção “Cadastrar”, o sistema solicita os dados necessários para criação da entidade (Paciente, Médico ou Funcionário). Após o preenchimento, a entidade passa pelo método *validar()*, definido pela interface *Validavel*. Caso seja válida, é adicionada ao Hospital por meio do gerenciador responsável. Caso seja inválida, o sistema exibe a mensagem de erro retornada por *getMensagemValidacao()* e o cadastro é cancelado.

c. Fluxo de Agendamento de Consulta

O usuário (Secretaria ou Administrador) escolhe a opção “Agendar consulta”. Em seguida, seleciona o paciente, o médico desejado e informa a data e horário da consulta. O sistema verifica se a data é válida e se não há conflito de horário para o médico. Caso tudo esteja correto, é criado um objeto *Consulta*, vinculando médico, paciente e horário, e ele é adicionado ao Hospital.

d. Fluxo de edição

Ao selecionar “Editar”, o usuário informa o ID da entidade que deseja modificar. O sistema localiza a entidade através do método *buscar()* do gerenciador correspondente. Se a entidade existir, são solicitados os novos dados. A nova versão passa pela validação via *Validavel*; se estiver correta, substitui a anterior usando o método *editar()*. Caso seja inválida, os dados antigos são mantidos e o sistema informa o erro.

e. Fluxo de busca

Ao escolher a opção “Buscar”, o usuário informa um identificador (geralmente o ID). O sistema utiliza o método *buscar()* do gerenciador para localizar a entidade. Se ela for encontrada, suas informações são exibidas de forma organizada conforme o tipo (paciente,

médico, funcionário ou consulta). Caso não exista, o sistema informa que nenhum registro foi localizado.

f. Fluxo de Listagem

Ao selecionar “Listar”, o sistema percorre todas as entidades do tipo escolhido (Pacientes, Médicos, Funcionários ou Consultas). O método *listar()* exibe cada item presente. No caso de classes que herdam de *Pessoa*, o sistema utiliza polimorfismo para chamar automaticamente o método *exibirInformacoes()*. Caso a lista esteja vazia, é exibida uma mensagem indicando que não há registros cadastrados.

g. Fluxo de Remoção

O usuário seleciona “Remover” e informa o ID da entidade desejada. O sistema busca o item e, se encontrado, o remove utilizando o método *remover()* do gerenciador. Uma confirmação é exibida para o usuário. Se o ID informado não existir, o sistema notifica que não foi possível realizar a remoção.

3.3 REGRAS DE NEGÓCIO

Para garantir a integridade e a segurança das informações, o sistema implementa as seguintes restrições:

- Validação de CPF: O CPF informado deve ser matematicamente válido. A verificação é realizada pela classe *Utilitarios*, que calcula os dígitos verificadores para impedir o cadastro de documentos fictícios ou com formatação incorreta.
- Formatação de CRM: O registro médico deve obedecer ao padrão nacional (sequência numérica, podendo conter a UF). A validação é feita via Expressões Regulares (Regex), garantindo que apenas formatos oficiais sejam aceitos.
- Consistência Temporal (Agendamento): Consultas só podem ser agendadas para datas e horários futuros. O sistema utiliza a biblioteca *java.time* para comparar o *input* do usuário com o momento atual, rejeitando agendamentos retroativos.
- Formato de Contato: O telefone deve conter apenas números e possuir entre 8 e 13 dígitos, assegurando que o dado possa ser utilizado para contato real.
- Privacidade de Dados Médicos: O sistema aplica um filtro de segurança onde médicos logados visualizam estritamente as consultas vinculadas às suas próprias credenciais, impedindo acesso à agenda de outros profissionais.

- **Segurança de Credenciais:** As senhas e nomes de usuário não podem ser nulos ou vazios. Essa verificação ocorre tanto no cadastro quanto na tentativa de *login* via *LoginException*.
- **Controle de Nível de Acesso:** O sistema restringe funcionalidades com base no *NivelAcesso* (Enum). Usuários tentam acessar apenas os menus permitidos para seu perfil (ex.: Médicos não acessam cadastro de funcionários), prevenindo uso indevido.
- **Integridade Obrigatória:** Nenhuma entidade (Paciente, Médico ou Consulta) é persistida na memória sem antes passar pelo método *validar()*. A implementação da interface *Validavel* atua como um "portão de entrada", rejeitando objetos incompletos ou inconsistentes antes que eles entrem nas listas do Hospital.

4. CONCLUSÃO

O projeto de Gerenciamento Hospitalar apresenta, na prática, como os conceitos de Programação Orientada a Objetos podem ser aplicados para construir um sistema seguro e bem estruturado. Por meio das funcionalidades implementadas, é possível manter a integridade das informações e gerenciar pacientes, profissionais e consultas de maneira clara e eficiente. Dessa forma, o sistema se consolida como uma base sólida, que não apenas cumpre seu propósito atual, mas também facilita futuras melhorias, expansões e manutenções.